

UNIVERSITÉ CHEIKH ANTA DIOP



INSTITUT CONFUCIUS



Année académique 2024 – 2025

Examen final L3

Module : Programmation Linux

Sujet :
Conception et développement d'un système d'administration
Linux automatisé

Réalisé par :
Elamine Ben ABDOU

Professeur :
M. Ousseynou

Résumé

Ce projet consiste en la conception d'une application d'administration système sous Linux, combinant des scripts Shell Bash et un programme en langage C. La partie Bash propose un menu interactif permettant d'automatiser les opérations courantes d'administration : gestion des répertoires et des fichiers, gestion des permissions, recherche, surveillance système, gestion des processus, sauvegarde automatique et planification de tâches. La partie C implémente un système de gestion d'inventaire informatique, permettant d'ajouter, modifier, supprimer, rechercher et afficher des équipements réseau, avec sauvegarde persistante des données dans un fichier.

L'ensemble du projet a été développé et versionné sur GitHub à l'aide des Workspaces (GitHub Codespaces), qui fournissent un environnement Ubuntu distant complet, prêt à l'emploi. Ce choix a permis de disposer nativement des outils Linux nécessaires (gcc, make, gdb) malgré un développement effectué sur un MacBook. Le projet a été compilé à l'aide d'un Makefile, débogué avec gdb, et l'historique complet du développement est disponible sur le dépôt GitHub.

I- Introduction

L'administration d'un système Linux repose sur un ensemble de tâches répétitives : création et organisation de fichiers, gestion des droits d'accès, surveillance des ressources, gestion des processus et sauvegarde des données. Automatiser ces opérations permet de gagner du temps, de réduire les erreurs humaines et d'assurer une meilleure traçabilité des actions effectuées sur le système.

Dans le cadre du module de Programmation Linux, ce projet vise à mettre en pratique les notions étudiées (Shell Bash, appels système, gestion des processus, programmation en C) à travers la réalisation d'une application complète d'administration système. Le rapport présente le contexte du projet, l'analyse des besoins, la conception retenue, la description technique des scripts Bash et du programme C, les difficultés rencontrées, ainsi que des captures d'écran illustrant le fonctionnement de l'application.

Le développement ayant été réalisé sur un ordinateur MacBook, ne disposant pas nativement d'un environnement Linux complet, l'utilisation des GitHub Workspaces a constitué une étape préalable indispensable, détaillée dans la section suivante.

II- Présentation de Linux

Linux est un système d'exploitation open source de type Unix, largement utilisé dans les environnements serveurs, embarqués et professionnels pour sa stabilité, sa sécurité et sa flexibilité. Il repose sur une architecture en couches : le noyau (kernel), qui gère les ressources matérielles (processeur, mémoire, périphériques, système de fichiers), et l'espace utilisateur, où s'exécutent les applications et les interpréteurs de commandes (shells).

Le Shell Bash est l'interpréteur de commandes le plus utilisé sous Linux. Il permet d'exécuter des commandes système, d'écrire des scripts pour automatiser des tâches, et d'interagir directement avec le noyau via des appels système. Le langage C, quant à lui, est le langage historique de développement système sous Linux : le noyau lui-même est écrit en C, ce qui permet un accès direct et performant aux appels système (fork, exec, fichiers, signaux, etc.), justifiant son usage dans la partie bas niveau de ce projet.

Utilisation des GitHub Workspaces (Codespaces)

Le développement de ce projet a été réalisé sur un MacBook, dont le système d'exploitation macOS, bien que dérivé d'Unix, ne fournit pas nativement l'ensemble des outils Linux requis par le projet (Makefile GNU, gdb, comportement exact de certaines commandes comme cron, at ou ps). Plutôt que d'installer une machine virtuelle ou de configurer un double-amorçage, le choix a été fait d'utiliser les GitHub Workspaces (Codespaces), qui permettent de lancer, directement depuis le dépôt GitHub, un environnement de développement distant basé sur une véritable distribution Ubuntu.

Cet environnement, accessible depuis le navigateur ou un éditeur relié au dépôt, fournit un terminal Linux complet avec gcc, make et gdb préinstallés ou installables via apt, ainsi qu'un accès direct au système de fichiers du projet versionné avec Git. Cela a permis de développer, compiler, déboguer et tester l'ensemble du projet dans des conditions Linux réelles, tout en conservant le confort de travail sur MacBook pour l'édition du code et la gestion du dépôt.

III- Analyse des besoins

L'analyse du cahier des charges a permis d'identifier les besoins fonctionnels suivants, répartis en deux grandes parties complémentaires :

Besoins fonctionnels : Partie Bash

- Gestion des répertoires : créer, supprimer, renommer, afficher le contenu
- Gestion des fichiers : créer, copier, déplacer, supprimer, rechercher
- Gestion des permissions : chmod, chown, affichage et modification des droits
- Recherche : fichier, texte dans un fichier, utilisateur, processus
- Surveillance système : mémoire, espace disque, utilisateurs connectés, charge CPU
- Gestion des processus : affichage, recherche, arrêt, envoi de signal
- Sauvegarde automatique : sauvegarde, compression, datation, restauration
- Planification : sauvegarde quotidienne via cron, restauration différée via at

Besoins fonctionnels : Partie C

- Ajout, modification, suppression et recherche d'un équipement informatique
- Affichage de l'ensemble des équipements enregistrés
- Persistance des données dans un fichier (fichier.txt / equipments.txt)

Besoins non fonctionnels

- Portabilité sous environnement Linux
- Compilation automatisée via Makefile
- Fiabilité du code, validée par débogage avec gdb
- Traçabilité du développement via Git (historique complet des commits)

IV- Conception

Le projet est organisé selon une arborescence claire séparant les scripts d'administration, le code source C, les données et la documentation, afin de faciliter la maintenance et la compréhension du projet :

```
ProjetLinux/  
├── README.md  
├── Makefile  
├── menu.sh  
├── sauvegarde.sh  
├── install.sh  
├── src/  
│   ├── main.c  
│   ├── fichier.c  
│   ├── fichier.h  
│   ├── equipement.c  
│   └── equipement.h  
├── data/  
│   └── equipements.txt  
├── backup/  
└── docs/  
    └── rapport.pdf
```

Le fichier menu.sh constitue le point d'entrée de la partie Bash : il affiche un menu interactif structuré en plusieurs sous-menus (répertoires, fichiers, permissions, recherche, surveillance, processus, sauvegarde, planification), chacun renvoyant vers des fonctions dédiées. Le script sauvegarde.sh centralise la logique de sauvegarde/restauration et est appelé à la fois manuellement depuis le menu et automatiquement via cron. Le script install.sh prépare l'environnement d'exécution (création des dossiers data/ et backup/, vérification des droits d'exécution des scripts).

Côté C, le programme suit une architecture modulaire :

- equipement.h définit la structure de données Equipement (identifiant, nom, adresse IP, adresse MAC, localisation, date d'ajout) ainsi que les prototypes des fonctions.
- equipement.c implémente la logique métier (CRUD) et la persistance dans le fichier equipements.txt.
- main.c gère l'interface en ligne de commande et l'aiguillage vers les différentes fonctionnalités.

Partie I : Description des scripts Bash

Le menu principal (menu.sh) propose une interface textuelle numérotée, permettant à l'utilisateur de naviguer entre les différentes catégories d'opérations à l'aide d'une boucle while associée à une structure case. Chaque choix invoque une fonction Bash dédiée.

1. Gestion des répertoires et des fichiers

Ces fonctions s'appuient sur les commandes système standards (mkdir, rm -r, mv, ls, touch, cp, find) encapsulées dans des fonctions Bash qui demandent à l'utilisateur les chemins nécessaires, vérifient leur existence avant toute action, et affichent un message de confirmation ou d'erreur.

2. Gestion des permissions

Les fonctions de gestion des permissions permettent de modifier les droits (chmod) et le propriétaire/groupe (chown) d'un fichier ou dossier, ainsi que d'afficher les permissions courantes via ls -l. Une vérification des droits de l'utilisateur (sudo si nécessaire) est intégrée pour les opérations chown.

3. Recherche

Le module de recherche combine plusieurs commandes : find pour localiser un fichier par son nom, grep pour rechercher une chaîne de texte à l'intérieur d'un fichier, id ou grep sur /etc/passwd pour rechercher un utilisateur, et ps combiné à grep pour rechercher un processus par son nom.

4. Surveillance du système

Les informations système sont obtenues via des commandes dédiées : free -h pour la mémoire, df -h pour l'espace disque, who ou w pour les utilisateurs connectés, et top ou uptime pour la charge processeur. Les résultats sont formatés pour une lecture claire dans le terminal.

5. Gestion des processus

Cette partie permet d'afficher la liste des processus actifs (ps aux), d'en rechercher un par nom, d'arrêter un processus via kill, et d'envoyer un signal spécifique (SIGTERM, SIGKILL, SIGSTOP, etc.) à un PID donné.

6. Sauvegarde automatique (sauvegarde.sh)

Le script de sauvegarde compresse un répertoire cible à l'aide de tar et gzip, en nommant l'archive avec un horodatage généré par la commande date, ce qui garantit l'unicité et la traçabilité de chaque sauvegarde. La fonction de restauration permet de décompresser une archive choisie vers son emplacement d'origine ou un dossier de destination.

7. Planification (cron et at)

Une tâche cron est configurée pour exécuter automatiquement le script de sauvegarde chaque jour à une heure définie (via crontab -e). La commande at est utilisée pour planifier une restauration ponctuelle différée, illustrant la différence entre planification récurrente (cron) et exécution unique différée (at).

Partie II : Description du programme C

Le programme C gère un inventaire d'équipements informatiques à travers une structure de données Equipement, définie dans equipment.h, regroupant : identifiant, nom, adresse IP, adresse MAC, localisation et date d'ajout.

```
typedef struct {  
    int id;  
    char nom[50];  
    char ip[16];  
    char mac[18];  
    char localisation[50];  
    char date_ajout[11];  
} Equipement;
```

Les équipements sont stockés dans un tableau (ou une liste chaînée) en mémoire pendant l'exécution, puis sauvegardés dans le fichier data/equipements.txt à l'aide des fonctions d'entrée/sortie standard du C, garantissant la persistance des données entre deux exécutions du programme.

Les fonctionnalités implémentées sont :

- ajouter_equipement() : saisie des informations et ajout au tableau/fichier
- modifier_equipement() : recherche par identifiant puis mise à jour des champs
- supprimer_equipement() : suppression par identifiant avec réécriture du fichier
- rechercher_equipement() : recherche par identifiant, nom ou adresse IP
- afficher_tous() : affichage formaté de tous les équipements enregistrés
- sauvegarder_fichier() : écriture de l'ensemble des données dans le fichier

Le programme propose un menu textuel en boucle (do...while) dans main.c, redirigeant vers les fonctions ci-dessus selon le choix de l'utilisateur. La compilation est automatisée par un Makefile qui définit les règles de compilation de chaque fichier .c en objet, puis leur édition de liens en exécutable, avec une cible clean pour supprimer les fichiers générés.

Le programme a été testé et débogué à l'aide de gdb, notamment pour vérifier la bonne gestion de la mémoire et des chaînes de caractères (dépassements de tampon, pointeurs nuls) lors des opérations de saisie et de recherche.

VII- Difficultés rencontrées

La principale difficulté rencontrée lors de la réalisation de ce projet a été d'ordre environnemental : le développement s'est fait sur un MacBook, dont le système d'exploitation macOS ne fournit pas nativement l'ensemble des outils Linux exigés par le sujet, notamment gdb et certaines versions ou comportements de make, ainsi que des commandes comme cron, at ou ps dont le fonctionnement diffère légèrement de celui de macOS par rapport à Linux.

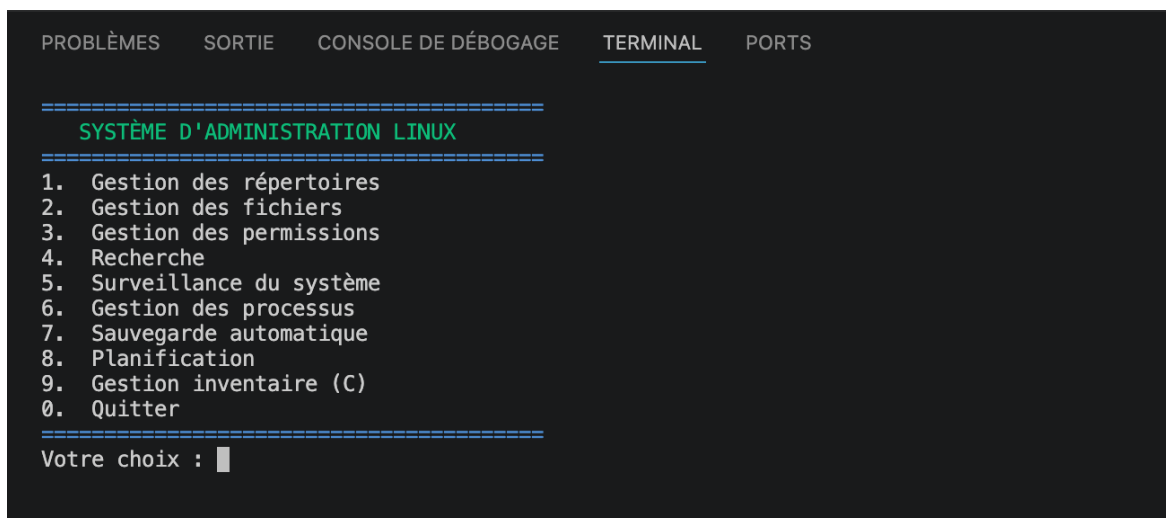
Pour résoudre ce problème sans avoir à installer une machine virtuelle ou un double système en local, la solution retenue a été d'utiliser les GitHub Workspaces (Codespaces), qui permettent de créer, directement depuis le dépôt GitHub, un environnement de développement distant basé sur une véritable image Ubuntu. Cela a demandé une phase d'adaptation initiale : configuration du Workspace, installation et vérification des paquets nécessaires (build-essential, gdb) via apt, ainsi que la prise en main du terminal distant intégré à l'éditeur relié au dépôt.

Une fois l'environnement Ubuntu correctement configuré via les Workspaces, l'ensemble des outils demandés (gcc, make, gdb, cron, at, Git) a fonctionné de manière fiable et conforme aux attentes du sujet, ce qui a permis de mener le projet à son terme dans de bonnes conditions.

VIII- Captures d'écran

Cette section illustre le fonctionnement de l'application à travers des captures d'écran prises lors de l'exécution du projet.

- Menu principal



```
PROBLÈMES  SORTIE  CONSOLE DE DÉBOGAGE  TERMINAL  PORTS

=====
SYSTÈME D'ADMINISTRATION LINUX
=====
1. Gestion des répertoires
2. Gestion des fichiers
3. Gestion des permissions
4. Recherche
5. Surveillance du système
6. Gestion des processus
7. Sauvegarde automatique
8. Planification
9. Gestion inventaire (C)
0. Quitter
=====
Votre choix : █
```

- Gestion des répertoires et des fichiers

```
PROBLÈMES  SORTIE  CONSOLE DE DÉBOGAGE  TERMINAL  PORTS  bash

--- GESTION DES RÉPERTOIRES ---
1. Créer un dossier
2. Supprimer un dossier
3. Renommer un dossier
4. Afficher le contenu
0. Retour
Choix : 3
Ancien : doss1
Nouveau : dssier2
renamed 'doss1' -> 'dssier2'
Appuyez sur Entrée...
```

```
PROBLÈMES  SORTIE  CONSOLE DE DÉBOGAGE  TERMINAL  PORTS  bash + ~

--- GESTION DES FICHIERS ---
1. Créer un fichier
2. Copier
3. Déplacer
4. Supprimer
5. Rechercher
0. Retour
Choix : 1
Nom : fic1
Appuyez sur Entrée...
```

- Gestion des permissions (chmod / chown)

```
PROBLÈMES  SORTIE  CONSOLE DE DÉBOGAGE  TERMINAL  PORTS  bash

--- GESTION DES PERMISSIONS ---
1. Afficher les permissions
2. chmod
3. chown
0. Retour
Choix : 1
Fichier : fic1
-rw-rw-rw- 1 codespace codespace 0 Jul 14 07:44 fic1
Appuyez sur Entrée...
```

```
PROBLÈMES  SORTIE  CONSOLE DE DÉBOGAGE  TERMINAL  PORTS  bash + ~

--- GESTION DES PERMISSIONS ---
1. Afficher les permissions
2. Modifier les droits
3. Modifier les permissions
0. Retour
Choix : 2
Fichier : fic1
Permissions : 755
mode of 'fic1' changed from 0666 (rw-rw-rw-) to 0755 (rwxr-xr-x)
Appuyez sur Entrée...
```

- Module de recherche

```

PROBLÈMES  SORTIE  CONSOLE DE DÉBOGAGE  TERMINAL  PORTS  bash

--- RECHERCHE ---
1. Fichier
2. Texte dans fichier
3. Utilisateur
4. Processus
0. Retour
Choix : 1
Nom : fic1
./fic1
Appuyez sur Entrée...

```

- Surveillance du système

```

PROBLÈMES  SORTIE  CONSOLE DE DÉBOGAGE  TERMINAL  PORTS  bash + - []

--- SURVEILLANCE ---
1. Mémoire
2. Espace disque
3. Utilisateurs
4. Charge CPU
0. Retour
Choix : 4
top - 08:07:10 up 1:25, 0 user, load average: 0.60, 0.46, 0.47
Tasks: 21 total, 1 running, 20 sleeping, 0 stopped, 0 zombie
%Cpu(s): 5.0 us, 15.0 sy, 10.0 ni, 70.0 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
MiB Mem : 7943.5 total, 422.1 free, 3038.4 used, 4863.1 buff/cache
MiB Swap: 0.0 total, 0.0 free, 0.0 used. 4905.2 avail Mem

  PID USER      PR  NI   VIRT   RES   SHR  S  %CPU  %MEM     TIME+ COMMAND
    1 codespa+  20   0   1136    568   516  S   0.0   0.0   0:00.12 docker--
    7 codespa+  20   0   2800   1844  1732  S   0.0   0.0   0:00.96 sh
   156 root      20   0  12020   4020  2880  S   0.0   0.0   0:00.02 sshd
Appuyez sur Entrée...

```

- Gestion des processus

```

PROBLÈMES  SORTIE  CONSOLE DE DÉBOGAGE  TERMINAL  PORTS  bash + - [] ... | {} x

--- PROCESSUS ---
1. Afficher tous
2. Rechercher
3. Arrêter (kill -15)
4. Envoyer un signal
0. Retour
Choix : 1
USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
codespa+  1  0.0  0.0   1136    568 ?        Ss   06:53   0:00 /sbin/docker-init -- /bin/sh -c echo Container started trap "
codespa+  7  0.0  0.0   2800   1844 ?        S    06:53   0:00 /bin/sh -c echo Container started trap "exit 0" 15 /usr/local
root     156  0.0  0.0  12020   4020 ?        Ss   06:53   0:00 sshd: /usr/sbin/sshd [listener] 0 of 10-100 startups
root     256  0.0  1.2 2033136 99004 ?        Sl   06:53   0:00 dockerd --dns 168.63.129.16
root     570  0.0  0.6 1803620 53772 ?        Ssl  06:53   0:01 containerd --config /var/run/docker/containerd/containerd.tom
codespa+ 1973  0.0  0.0   2800   1980 ?        Ss   06:54   0:00 /bin/sh
root     2010  0.0  0.0   2800   1924 ?        Ss   06:54   0:00 /bin/sh
codespa+ 2159  0.0  0.0   2808   1992 ?        Ss   06:54   0:00 sh /home/codespace/.vscode-remote/bin/4fe60c8b1cdac1c4c174f2f
codespa+ 2168  0.2  2.0 1800820 165688 ?        Sl   06:54   0:09 /vscode/bin/linux-x64/4fe60c8b1cdac1c4c174f2fb180d0d758272d71
codespa+ 2210  1.0  4.6 18769596 375608 ?        Sl   06:54   0:49 /vscode/bin/linux-x64/4fe60c8b1cdac1c4c174f2fb180d0d758272d71
codespa+ 2226  0.0  0.7 1659324 59032 ?        Sl   06:54   0:00 /vscode/bin/linux-x64/4fe60c8b1cdac1c4c174f2fb180d0d758272d71
codespa+ 2764  0.1  0.9 1536956 78048 ?        Sl   06:54   0:04 /vscode/bin/linux-x64/4fe60c8b1cdac1c4c174f2fb180d0d758272d71
codespa+ 2871  0.0  0.9 1599852 78484 ?        Sl   06:54   0:00 /vscode/bin/linux-x64/4fe60c8b1cdac1c4c174f2fb180d0d758272d71
codespa+ 3321  0.0  1.0 9982856 86988 ?        Sl   06:54   0:00 /vscode/bin/linux-x64/4fe60c8b1cdac1c4c174f2fb180d0d758272d71
codespa+ 3824  0.0  0.0   2800   1988 ?        Ss   06:54   0:00 /bin/sh
root     3864  0.0  0.0   2800   1840 ?        Ss   06:54   0:00 /bin/sh
codespa+ 4349  0.0  0.1 15960   12408 pts/0    Ss   06:55   0:00 /bin/bash --init-file /vscode/bin/linux-x64/4fe60c8b1cdac1c4c
codespa+ 29112 0.0  0.0   8008   3848 pts/0    S+   07:51   0:00 /bin/bash ./menu.sh
codespa+ 37292 0.0  0.0   6112   2088 ?        S    08:09   0:00 sleep 1
codespa+ 37293 0.0  0.0  11324   4436 pts/0    R+   08:09   0:00 ps aux
Appuyez sur Entrée...

```

- Sauvegarde et restauration

```
PROBLÈMES  SORTIE  CONSOLE DE DÉBOGAGE  TERMINAL  PORTS  bash + - [ ] [X]

=== SAUVEGARDE AUTOMATIQUE ===
Répertoire à sauvegarder : dossier1
Type de compression :
1) tar.gz
2) zip
Votre choix : 1
tar (child): backup/backup_dossier1_20260714_081052.tar.gz: Cannot open: No such file or directory
tar (child): Error is not recoverable: exiting now
tar: Child returned status 2
tar: Error is not recoverable: exiting now
Sauvegarde créée : backup/backup_dossier1_20260714_081052.tar.gz
Appuyez sur Entrée...
```

- Planification avec cron et at

```
PROBLÈMES  SORTIE  CONSOLE DE DÉBOGAGE  TERMINAL  PORTS  bash + - [ ] [X]

--- PLANIFICATION ---
1. Cron (sauvegarde quotidienne)
2. At (restauration)
0. Retour
Choix : 1
Commande : crontab -l
./menu.sh: line 167: crontab: command not found
Ajouté à cron (2h)
Appuyez sur Entrée...
```

- Programme C : Gestion d'inventaire

```
PROBLÈMES  SORTIE  CONSOLE DE DÉBOGAGE  TERMINAL  PORTS  bash + - [ ] [X] ...

--- INVENTAIRE (C) ---
gcc -Wall -Wextra -g -c src/main.c -o src/main.o
gcc -Wall -Wextra -g -c src/fichier.c -o src/fichier.o
gcc -Wall -Wextra -g -c src/equipement.c -o src/equipement.o
gcc -Wall -Wextra -g -o src/main src/main.o src/fichier.o src/equipement.o

=== GESTION INVENTAIRE ===
1. Ajouter un équipement
2. Modifier un équipement
3. Supprimer un équipement
4. Rechercher un équipement
5. Afficher tous les équipements
6. Sauvegarder
0. Quitter
Votre choix : 1
ID : 05
Nom : Ben_abdou
Adresse IP : 192.168.1.14
Adresse MAC : 4A:6F:3G:5H:3C
Localisation : sahm1
✓ Équipement ajouté
✓ Sauvegarde effectuée (5 équipements)
```

- Compilation et débogage

```
PROBLÈMES  SORTIE  CONSOLE DE DÉBOGAGE  TERMINAL  PORTS
@280903-code → /workspaces/projet_final_linux (main) $ gdb ./bin/inventaire
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./bin/inventaire...
(gdb) break main
Breakpoint 1 at 0x1151: file src/c/main.c, line 3.
(gdb) run
Starting program: /workspaces/projet_final_linux/bin/inventaire

This GDB supports auto-downloading debuginfo from the following URLs:
<https://debuginfod.ubuntu.com>
Enable debuginfod for this session? (y or [n]) y
Debuginfod has been enabled.
To make this setting permanent, add 'set debuginfod enabled on' to .gdbinit.
Downloading separate debug info for system-supplied DSO at 0x7ffff7fc3000
Download failed: Timer expired. Continuing without separate debug info for system-supplied DSO at 0x7ffff7fc3000.
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, main () at src/c/main.c:3
3   printf("Inventaire C - version demo\n");
(gdb) next
Inventaire C - version demo
4   return 0;
(gdb) print argc
No symbol "argc" in current context.
(gdb) continue
Continuing.
[Inferior 1 (process 52526) exited normally]
(gdb) █
```

- Historique Git

```
● @280903-code → /workspaces/projet_final_linux (main) $ git log --oneline
046d6ed (HEAD -> main) ajout du fichier main.c pour tester la compilation
32d025d (origin/main, origin/HEAD) Ajout du .gitignore pour exclure les fichiers générés
dff39fb Ajout des scripts Bash
e043ec2 Création de l'arborescence et fichiers de base
○ @280903-code → /workspaces/projet_final_linux (main) $ █
```

IX- Conclusion

Ce projet a permis de mettre en application concrète les notions fondamentales de l'administration système sous Linux, en combinant la puissance du Shell Bash pour l'automatisation des tâches courantes et la rigueur du langage C pour la gestion structurée des données. La réalisation d'un menu interactif complet, couplé à un système de sauvegarde planifiée et à un programme de gestion d'inventaire, illustre la complémentarité entre scripting système et programmation bas niveau.

Ce travail a également été l'occasion de se familiariser avec les outils professionnels de développement (Makefile, gdb, Git), indispensables pour tout projet d'administration système sérieux. Les perspectives d'amélioration incluent l'ajout d'une interface graphique, la centralisation des logs d'exécution, ou encore le renforcement de la sécurité.

Bibliographie

- Documentation GNU Make : <https://www.gnu.org/software/make/manual/> (consulté le 13 juillet 2026)
- Documentation sur Git Grafikart : <https://grafikart.fr/formations> (consulté le 10 juillet 2026)
- Documentation Github : <https://docs.github.com/fr> (consulté le 09 juillet 2026)
- Information linux et mac : <https://openclassrooms.com/fr/> (consulté le 09 juillet 2026)